CS 112

Introduction to Data Structures

WEEK 10

Recursion and Sorting

Eric Araújo

Calvin University · Fall 2026

Pete and Repeat

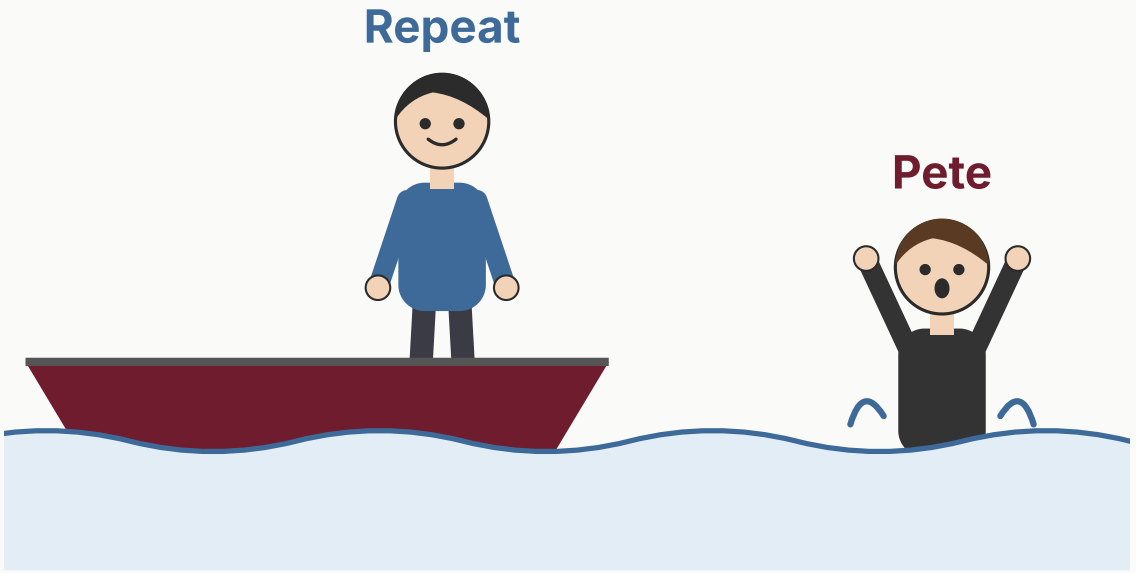
Pete and Repeat were in a boat.

Pete fell out.

Who was left?

Repeat.

↻ Pete and Repeat were in a boat. Pete fell out. Who was left?...



The story tells itself again, and nothing in it says stop.

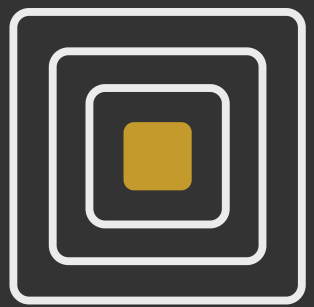
This Week

1. What recursion is
2. Base case and recursive step
3. The run-time stack KEY
4. Towers of Hanoi
5. Selection sort and insertion sort
6. Merge sort: sorting by recursion

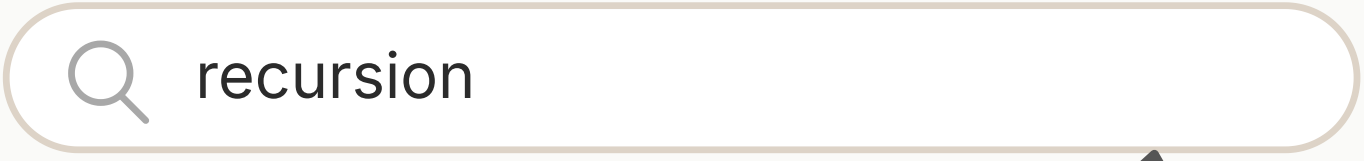
PART 1 OF 4

Recursion

solve it by solving a smaller one



Ask a Search Engine



Did you mean: ***recursion***

click it: the same search, again

Try it tonight: search for **recursion**.

Recursive Names

```
GNU
GNU's Not Unix
(GNU's Not Unix)'s Not Unix
((GNU's Not Unix)'s Not Unix)'s Not Unix
… and it never ends
```

WINE = **Wine** Is Not an Emulator

PHP = **PHP**: Hypertext Preprocessor

One Sentence, Two Endings



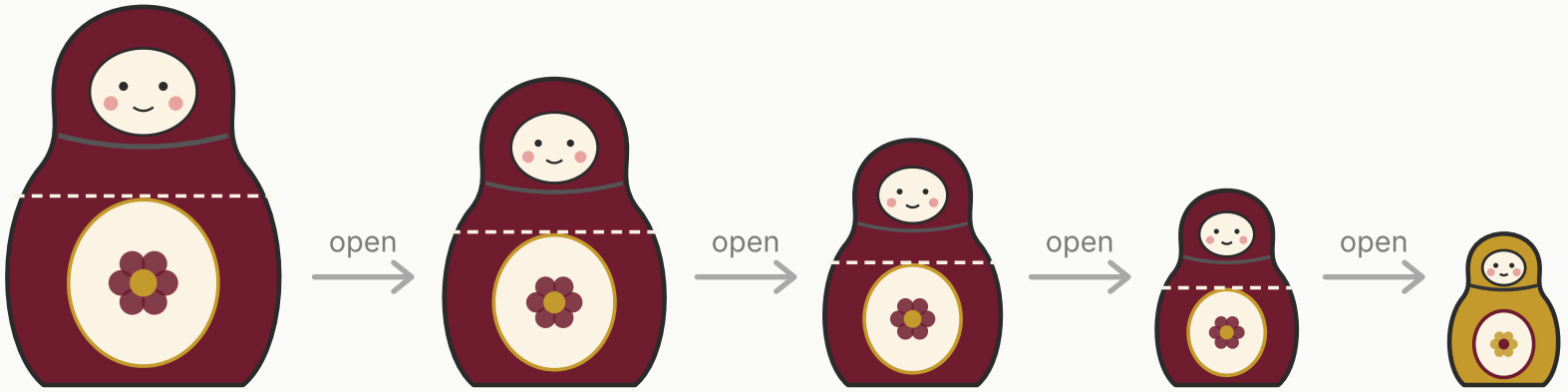
To understand recursion, you must first understand recursion.



To understand recursion, you must first understand recursion, **unless you already do. Then stop.**

the base case

The Same Thing, Smaller



solid: does not open

Open the doll, find a smaller doll, until one is too small to open.

A Function That Calls Itself

The sum of an array is the first item plus **the sum of the rest**.

```
int sum(int a[], int size) {
    if (size == 0) {
        return 0;           // no items: the sum is 0
    }
    return a[0] + sum(&a[1], size - 1);  // first + the rest
}
```

&a[1] is the array starting one item later: the rest.

How to Write One

1. The base case

An instance so small the answer needs no work.
Often `n == 0` or `n == 1`.

2. The recursive step

Solve the general case by **using** the answer to a smaller one.
For `f(n)`, often `f(n - 1)`.

Every call must get **closer** to the base case.

Factorial

$$n! = n \times (n - 1)! \qquad 0! = 1! = 1$$

```
unsigned factorial(unsigned n) {
    if (n <= 1) {
        return 1;                       // base case
    }
    return n * factorial(n - 1);        // recursive step
}
```

TALK TO YOUR NEIGHBOR · TTYN

`main` calls `factorial(4)`. How many times is `factorial` called, in total?

A. 1

B. 3

C. 4

D. 5

TALK TO YOUR NEIGHBOR · TTYN

The base case is gone. What happens when `main` calls `factorial(3)`?

```
unsigned factorial(unsigned n) {
    return n * factorial(n - 1);
}
```

A. It returns 0

B. It calls itself until the program runs out of stack and crashes

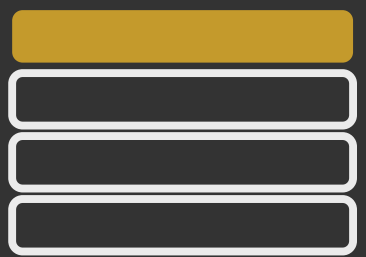
C. It does not compile

D. It returns 6, as before

PART 2 OF 4

The Run-Time Stack

one frame per call, last in, first out

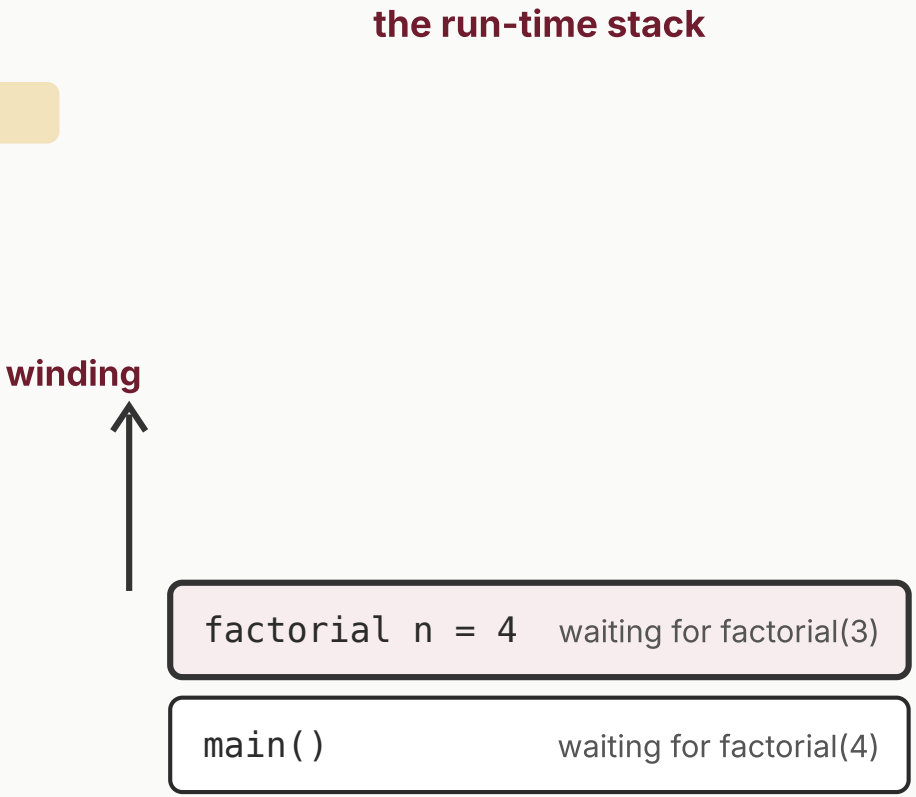


Tracing factorial(4)

1. main calls factorial(4): a frame for it goes on the stack.

```
// main
cout << factorial(4);

unsigned factorial(unsigned n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

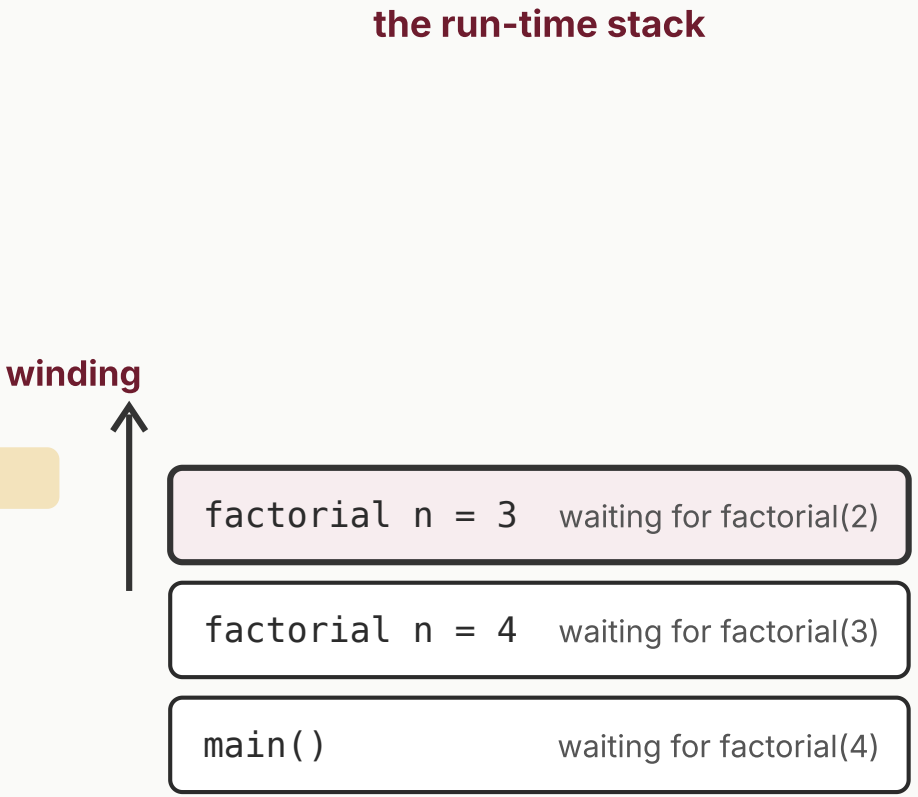


Tracing factorial(4)

2 n is 4, not 1, so it calls factorial(3), and waits.

```
// main
cout << factorial(4);

unsigned factorial(unsigned n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

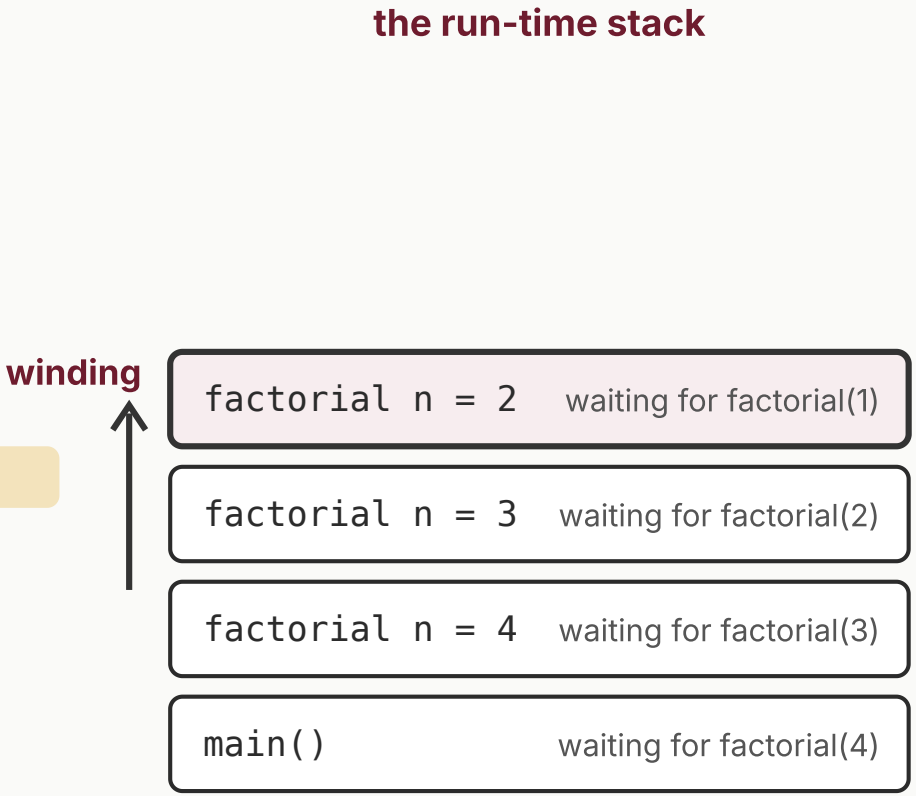


Tracing factorial(4)

3 factorial(3) calls factorial(2), and waits.

```
// main
cout << factorial(4);

unsigned factorial(unsigned n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

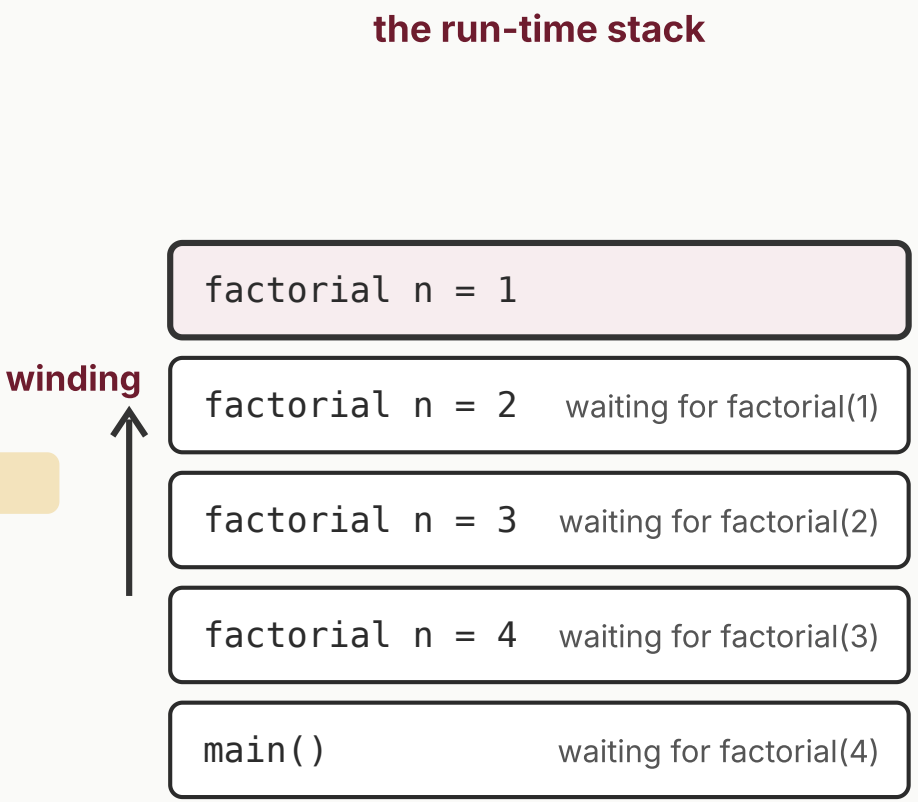


Tracing factorial(4)

4 factorial(2) calls factorial(1), and waits.

```
// main
cout << factorial(4);

unsigned factorial(unsigned n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

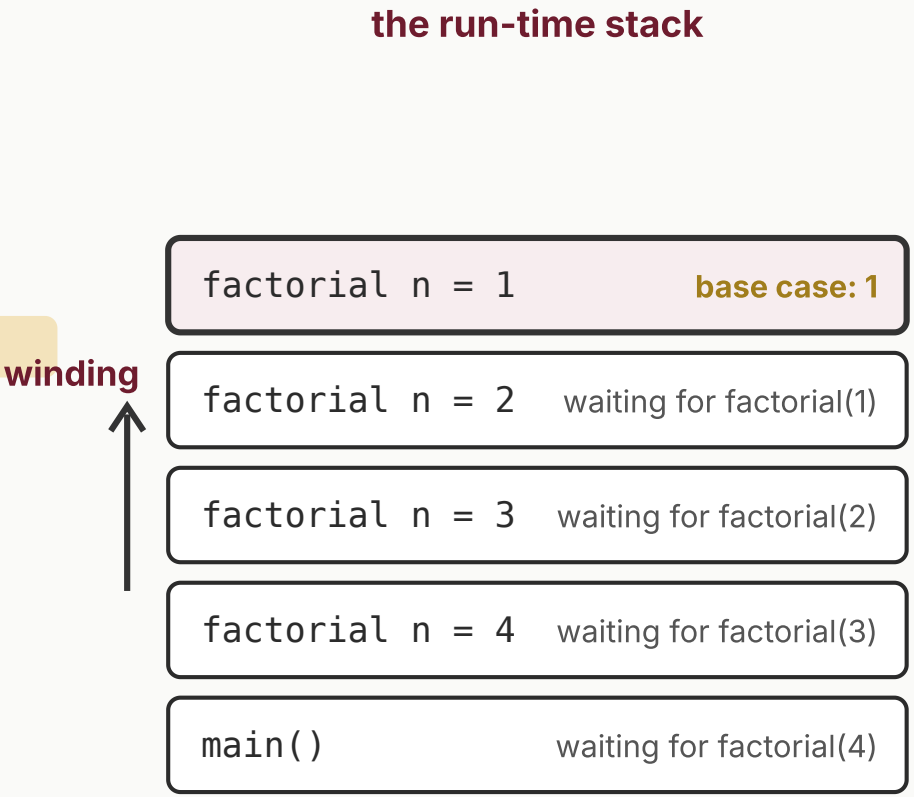


Tracing factorial(4)

5 n is 1: the base case. It returns 1. Winding is over.

```
// main
cout << factorial(4);

unsigned factorial(unsigned n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```



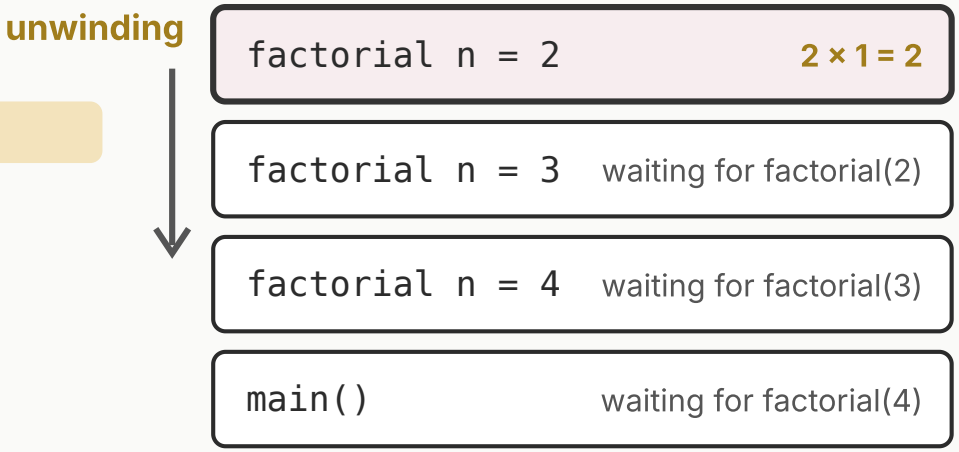
Tracing factorial(4)

6 factorial(1) is popped. factorial(2) gets its 1 and returns 2 × 1 = 2.

```
// main
cout << factorial(4);

unsigned factorial(unsigned n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

the run-time stack

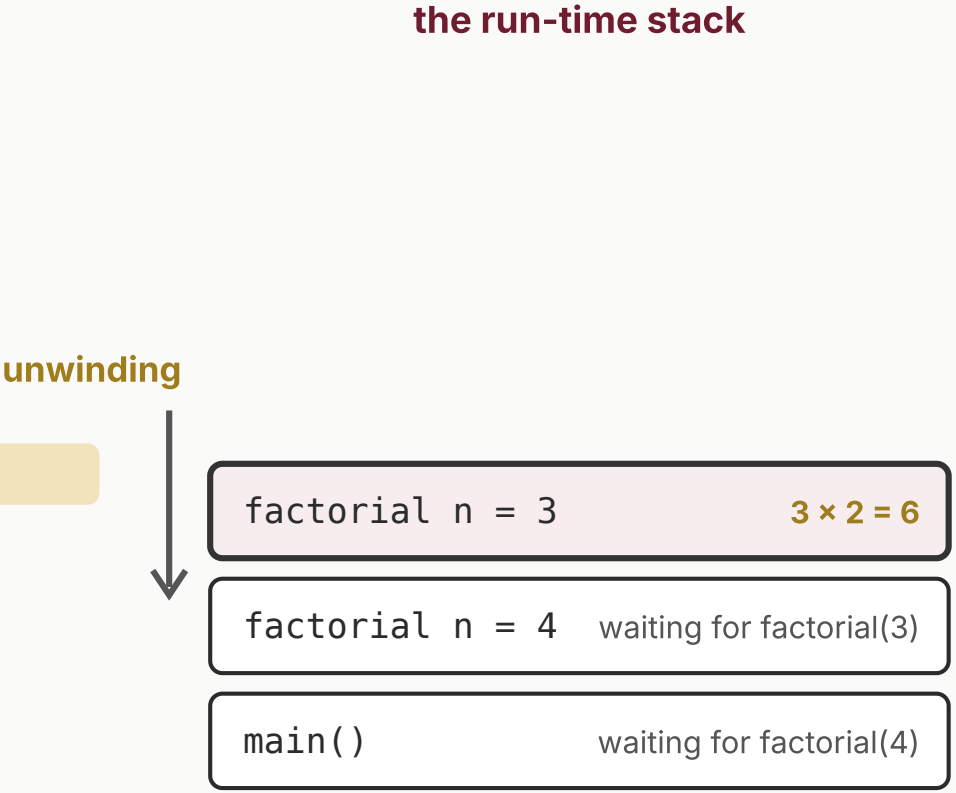


Tracing factorial(4)

7 factorial(2) is popped. factorial(3) returns 3 × 2 = 6.

```
// main
cout << factorial(4);

unsigned factorial(unsigned n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```



Tracing factorial(4)

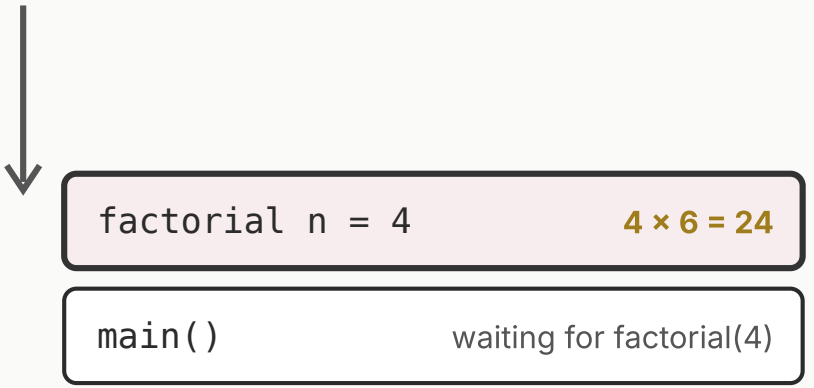
8 factorial(3) is popped. factorial(4) returns 4 × 6 = 24.

```
// main
cout << factorial(4);

unsigned factorial(unsigned n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

the run-time stack

unwinding



Tracing factorial(4)

9 factorial(4) is popped. main gets 24 and prints it.

```
// main
cout << factorial(4);

unsigned factorial(unsigned n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

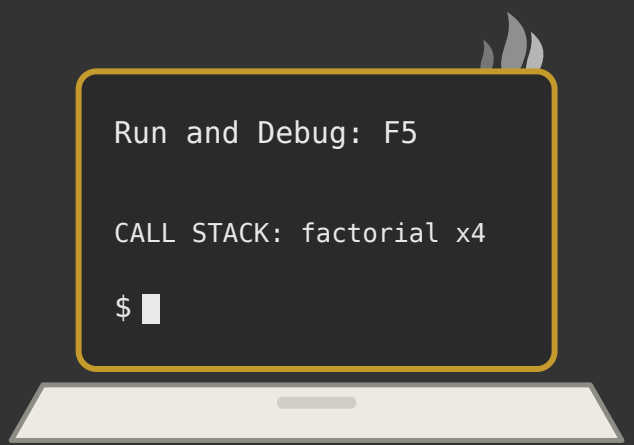
the run-time stack

unwinding





Demo



factorial(4) in the VS Code debugger: the Call Stack panel, one frame at a time.

step in, watch the frames pile up, step out

TALK TO YOUR NEIGHBOR · TTYN

What is the time complexity of `factorial(n)`?

A. O(1)

B. O(lg n)

C. O(n)

D. $O(n^2)$

Recursion Is Not Free

Recursive

```
int sum(int a[], int size) {
    if (size == 0) {
        return 0;
    }
    return a[0] + sum(&a[1], size - 1);
}
```

O(n) time, **n frames** on the stack

Iterative

```
int sum(int a[], int size) {
    int result = 0;
    for (int i = 0; i < size; ++i) {
        result += a[i];
    }
    return result;
}
```

O(n) time, **one frame**

When Recursion Is the Better Tool

Print a singly-linked list **backwards**. A loop has to walk from the front for every item: O(n²).

```
template<class Item>
void List<Item>::Node::printReverse(ostream& out) const {
    if (myNext != nullptr) {
        myNext->printReverse(out);    // the nodes behind me first
    }
    out << myItem << ' ';             // then me
}
```

One pass, O(n). The run-time stack remembers the way back.

PART 3 OF 4

Towers of Hanoi

three pegs, n disks, one rule



Towers of Hanoi



Move every disk from **A** to **B**, one at a time.
Never put a larger disk on a smaller one.

Can We Spot a Pattern?

2 disks

```
move(1, src, aux, dest);
move(1, src, dest, aux);
move(1, aux, dest, src);
```

3 disks

```
move(2, src, aux, dest);
move(1, src, dest, aux);
move(2, aux, dest, src);
```

4 disks

```
move(3, src, aux, dest);
move(1, src, dest, aux);
move(3, aux, dest, src);
```

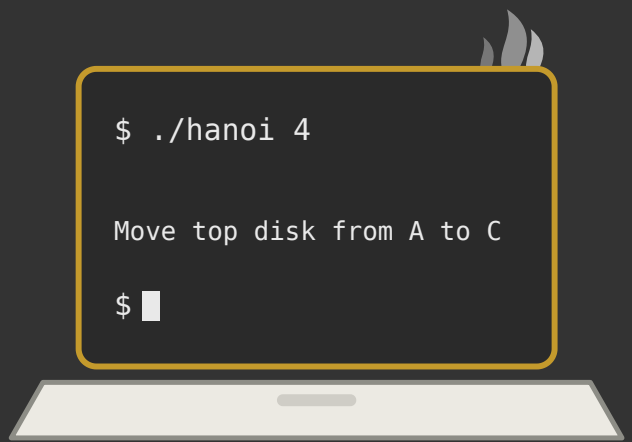
```
move(n - 1, src, aux, dest);  // the top n - 1, out of the way
move(1, src, dest, aux);        // the biggest, to its place
move(n - 1, aux, dest, src);  // the n - 1, back on top
```

Hanoi, Recursively

```
void move(int numDisks, char src, char dest, char aux) {
    if (numDisks == 1) {                                    // base case
        cout << "Move top disk from " << src << " to " << dest << endl;
        return;
    }
    move(numDisks - 1, src, aux, dest);   // the smaller tower, out of the way
    move(1, src, dest, aux);              // the biggest disk
    move(numDisks - 1, aux, dest, src);   // the smaller tower, back on top
}

int main() {
    move(4, 'A', 'B', 'C');
}
```

Demo



Four disks, by hand. Then move(4, 'A', 'B', 'C') in code.

count the moves

TALK TO YOUR NEIGHBOR · TTYN

How many lines does `move(4, 'A', 'B', 'C')` print?

A. 4

B. 8

C. 15

D. 16

Sixty-Four Disks

moves

$2^{64} - 1$

$\approx 1.8 \times 10^{19}$

a computer printing

10^{6} / s

a million moves a second

still printing after

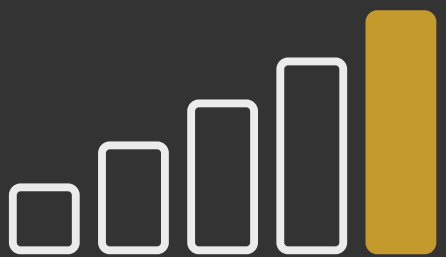
585,000

years

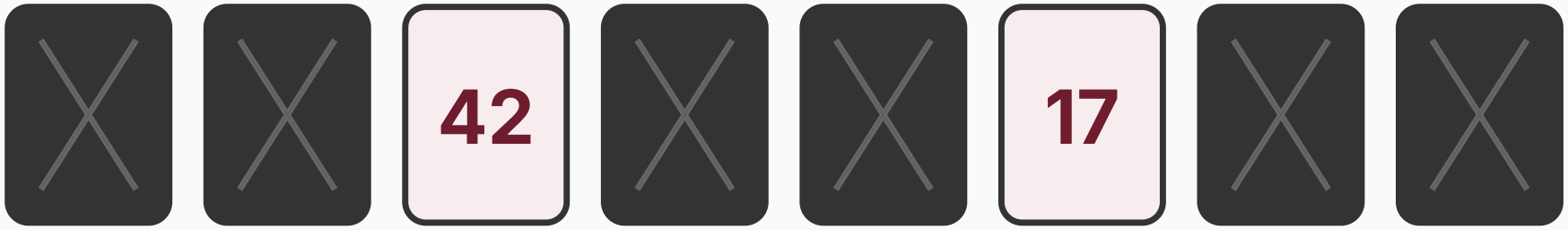
PART 4 OF 4

Sorting

put them in order, two at a time



Sort It by Hand

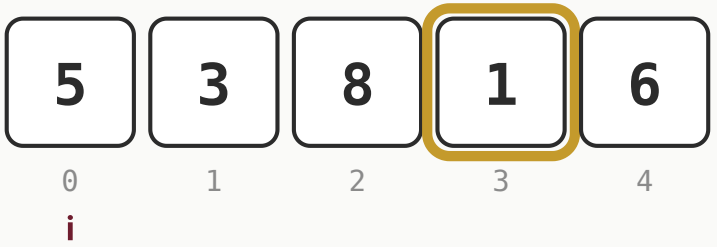


1. Write 8 different numbers on a sheet. Tear it into 8 pieces. Shuffle.
2. Lay them face down in a row, like an array.
3. Sort them. Turn over **two at a time**, never more.
4. Write down, step by step, what you did.

Selection Sort

1. i = 0: the smallest of 5 3 8 1 6 is 1, at index 3.

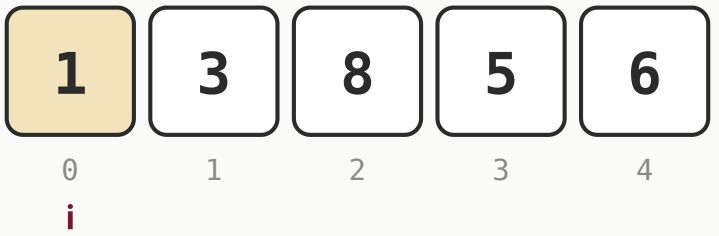
```
for (int i = 0; i < n - 1; ++i) {
    int smallest = i;
    for (int j = i + 1; j < n; ++j) {
        if (a[j] < a[smallest]) {
            smallest = j;
        }
    }
    swap(a[i], a[smallest]);
}
```



Selection Sort

2 Swap it to index 0. 1 is in its final place.

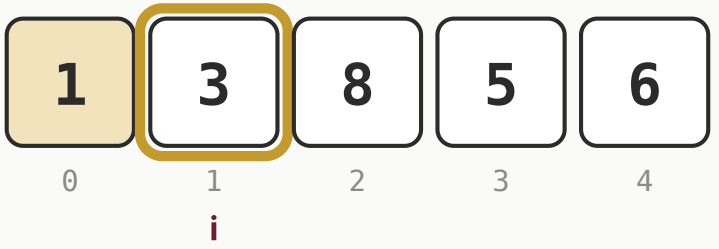
```
for (int i = 0; i < n - 1; ++i) {
    int smallest = i;
    for (int j = i + 1; j < n; ++j) {
        if (a[j] < a[smallest]) {
            smallest = j;
        }
    }
    swap(a[i], a[smallest]);
}
```



Selection Sort

3 i = 1: the smallest of 3 8 5 6 is 3, already at index 1.

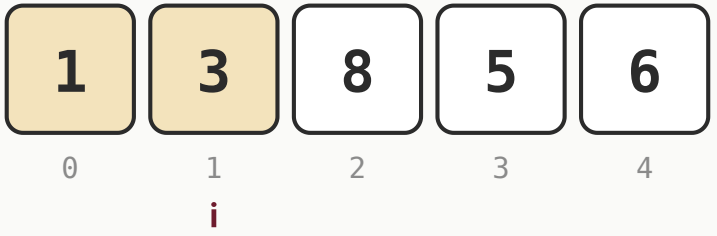
```
for (int i = 0; i < n - 1; ++i) {
    int smallest = i;
    for (int j = i + 1; j < n; ++j) {
        if (a[j] < a[smallest]) {
            smallest = j;
        }
    }
    swap(a[i], a[smallest]);
}
```



Selection Sort

4 It swaps with itself. 1 3 are in place.

```
for (int i = 0; i < n - 1; ++i) {
    int smallest = i;
    for (int j = i + 1; j < n; ++j) {
        if (a[j] < a[smallest]) {
            smallest = j;
        }
    }
    swap(a[i], a[smallest]);
}
```



Selection Sort

5 i = 2: the smallest of 8 5 6 is 5, at index 3.

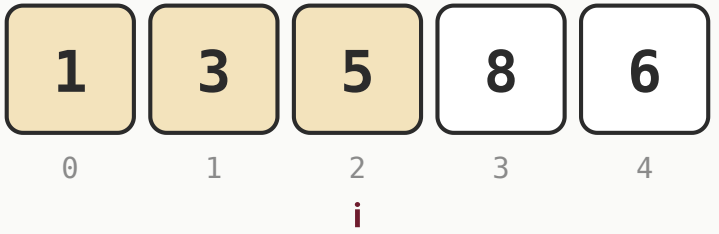
```
for (int i = 0; i < n - 1; ++i) {
    int smallest = i;
    for (int j = i + 1; j < n; ++j) {
        if (a[j] < a[smallest]) {
            smallest = j;
        }
    }
    swap(a[i], a[smallest]);
}
```



Selection Sort

6 Swap: 1 3 5 are in place.

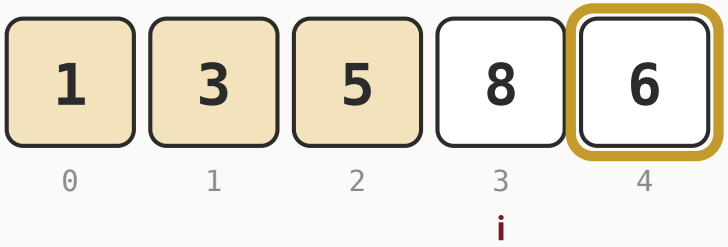
```
for (int i = 0; i < n - 1; ++i) {
    int smallest = i;
    for (int j = i + 1; j < n; ++j) {
        if (a[j] < a[smallest]) {
            smallest = j;
        }
    }
    swap(a[i], a[smallest]);
}
```



Selection Sort

7 i = 3: the smallest of 8 6 is 6, at index 4.

```
for (int i = 0; i < n - 1; ++i) {
    int smallest = i;
    for (int j = i + 1; j < n; ++j) {
        if (a[j] < a[smallest]) {
            smallest = j;
        }
    }
    swap(a[i], a[smallest]);
}
```



Selection Sort

8 Swap. The last item is in place too: sorted.

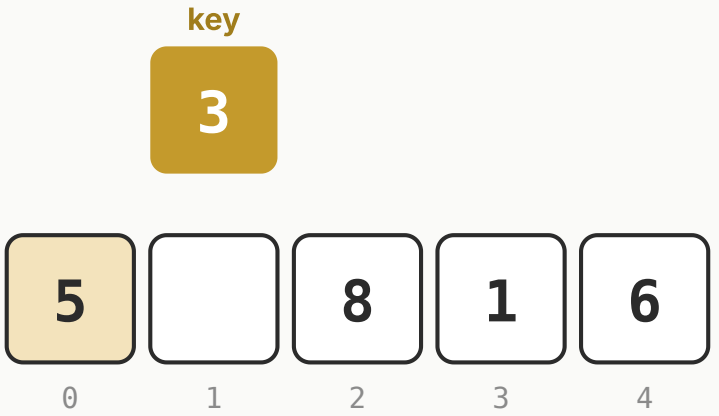
```
for (int i = 0; i < n - 1; ++i) {
    int smallest = i;
    for (int j = i + 1; j < n; ++j) {
        if (a[j] < a[smallest]) {
            smallest = j;
        }
    }
    swap(a[i], a[smallest]);
}
```



Insertion Sort

1 i = 1: lift out 3. The sorted part is just 5.

```
for (int i = 1; i < n; ++i) {
    int key = a[i];
    int j = i - 1;
    while (j >= 0 && a[j] > key) {
        a[j + 1] = a[j];
        j = j - 1;
    }
    a[j + 1] = key;
}
```



Insertion Sort

2 5 is bigger than 3, so it slides right; 3 drops into index 0.

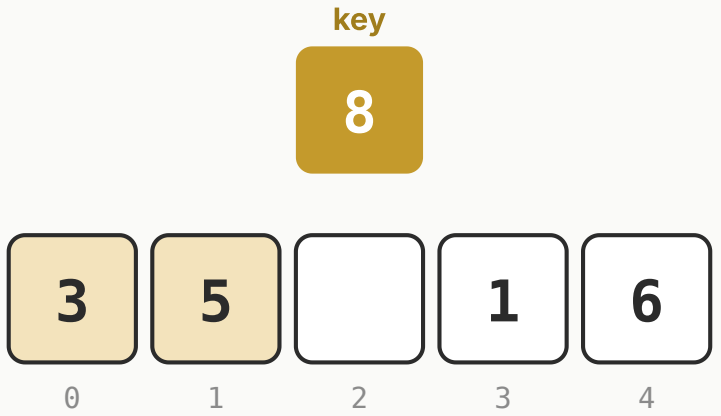
```
for (int i = 1; i < n; ++i) {
    int key = a[i];
    int j = i - 1;
    while (j >= 0 && a[j] > key) {
        a[j + 1] = a[j];
        j = j - 1;
    }
    a[j + 1] = key;
}
```



Insertion Sort

3 i = 2: lift out 8.

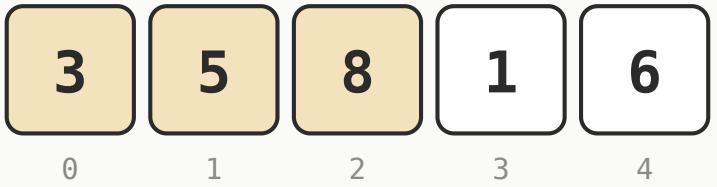
```
for (int i = 1; i < n; ++i) {
    int key = a[i];
    int j = i - 1;
    while (j >= 0 && a[j] > key) {
        a[j + 1] = a[j];
        j = j - 1;
    }
    a[j + 1] = key;
}
```



Insertion Sort

4 Nothing in 3 5 is bigger than 8: it drops straight back.

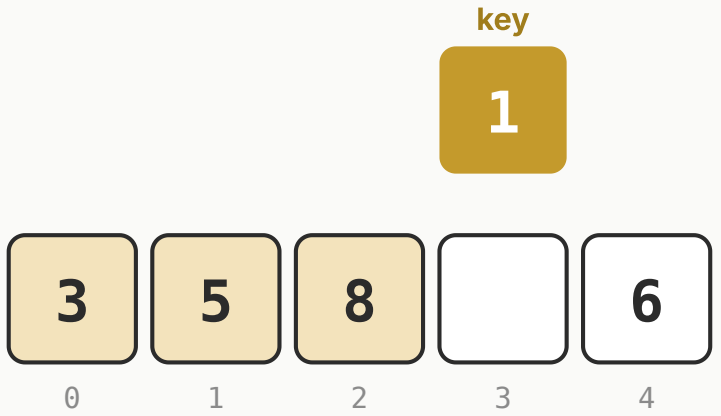
```
for (int i = 1; i < n; ++i) {
    int key = a[i];
    int j = i - 1;
    while (j >= 0 && a[j] > key) {
        a[j + 1] = a[j];
        j = j - 1;
    }
    a[j + 1] = key;
}
```



Insertion Sort

5 i = 3: lift out 1.

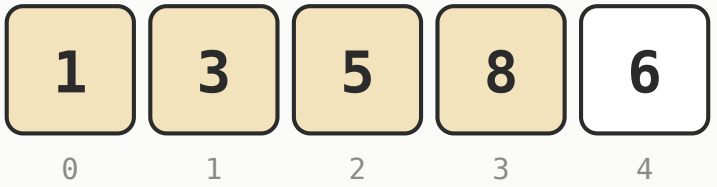
```
for (int i = 1; i < n; ++i) {
    int key = a[i];
    int j = i - 1;
    while (j >= 0 && a[j] > key) {
        a[j + 1] = a[j];
        j = j - 1;
    }
    a[j + 1] = key;
}
```



Insertion Sort

6 8, 5 and 3 all slide right; 1 drops into index 0.

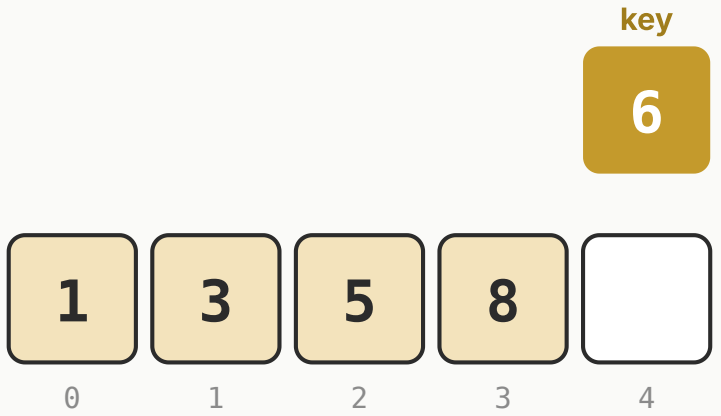
```
for (int i = 1; i < n; ++i) {
    int key = a[i];
    int j = i - 1;
    while (j >= 0 && a[j] > key) {
        a[j + 1] = a[j];
        j = j - 1;
    }
    a[j + 1] = key;
}
```



Insertion Sort

7 i = 4: lift out 6.

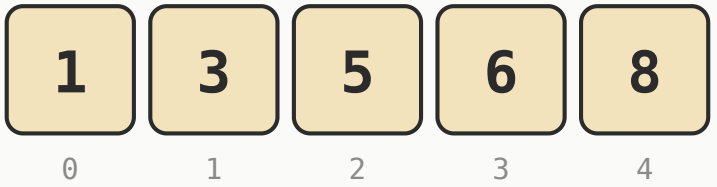
```
for (int i = 1; i < n; ++i) {
    int key = a[i];
    int j = i - 1;
    while (j >= 0 && a[j] > key) {
        a[j + 1] = a[j];
        j = j - 1;
    }
    a[j + 1] = key;
}
```



Insertion Sort

8 8 slides right; 6 drops into index 3. Sorted.

```
for (int i = 1; i < n; ++i) {
    int key = a[i];
    int j = i - 1;
    while (j >= 0 && a[j] > key) {
        a[j + 1] = a[j];
        j = j - 1;
    }
    a[j + 1] = key;
}
```



TALK TO YOUR NEIGHBOR · TTYN

The array is `1 2 3 4 6 5`. Which sort finishes with fewer comparisons?

A. Selection sort

B. Insertion sort

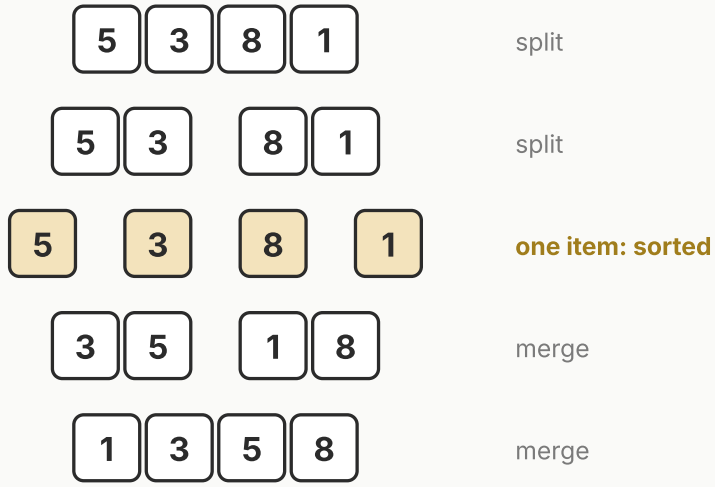
C. The same number

The Two, Side by Side

	Selection sort	Insertion sort
Best case	$O(n^2)$	**O(n)**, already sorted
Worst case	$O(n^2)$	$O(n^2)$
Extra memory	O(1)	O(1)
Stable: equal items keep their order	no	**yes**

Merge Sort

```
void mergeSort(int a[], int n) {
    if (n <= 1) {
        return;              // base case
    }
    int half = n / 2;
    mergeSort(a, half);                 // left half
    mergeSort(&a[half], n - half);      // right half
    merge(a, half, n);                  // merge them
}
```



n = 1,000,000	selection, insertion: n^2	merge sort: n lg n
steps	10^{12}	2×10^7

Demo



Selection sort, insertion sort and std::sort, on 50,000 random numbers.

guess each time before it runs

Week 10 Recap

- A recursive function needs a **base case** and a step that gets **closer** to it
- Every call gets a frame on the **run-time stack**
- The work happens as the frames come back off: **unwinding**
- Towers of Hanoi: $2^n - 1$ moves, $O(2^n)$
- Selection and insertion sort: $O(n^2)$; insertion sort is $O(n)$ on sorted input
- Merge sort recurses: $O(n \lg n)$

Next: a12 · a13 →

This deck stands on earlier CS112 materials by
Joel Adams and **Victor Norman** · adapted and extended by **Eric Araújo**